HW5:Clustering

Savannah

2025-10-21

Libraries Needed:

```
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

```
library(cluster)
library(glue)
library(gridExtra)
library(factoextra)
```

```
## Warning: package 'factoextra' was built under R version 4.4.3
```

#Task 1: Clusters and the Gap Statistic

Data Loading:

From the data_generation.py file, I generated the necessary datasets according to the directions on the PDF. I will now proceed to load all datasets.

```
file_list <- list.files("derived_data_1", pattern = "*.csv", full.names = TRUE)

# Read all CSVs into a named list of data frames
df_list <- file_list %>%
  set_names(~ basename(.) %>% tools::file_path_sans_ext()) %>%
  map(read_csv)

n <- c(2:6)
side_length <- c(1,10,2:9)
```

Clustergap Simulations:

For each data set, I will run the cluster gap function. Then, I will visualize the results. The code chunk below will plot the clustergap results for each individual dataset. I have chosen not to run it, but if plots are needed, then they can be run.

```r
# to save these plots:
if (!dir.exists("figures_1")) {
  dir.create("figures_1")
}
#creation of plots:

for (df_name in names(df_list)) {
  df <- df_list[[df_name]]
  cg <- clusGap(df %>% as.matrix(), kmeans, nstart = 20, iter.max = 50, K.max = 11, B = 10)

  # Define file path
  filename <- file.path("figures", paste0("gap_plot_", df_name, ".png"))

  # Open PNG device
  png(filename, width = 800, height = 600)

  # Create the plot
  plot(cg,
       main = glue("Cluster Gap for Dataset {df_name}"),
       xlab = "Number of Clusters")

  # Close the plotting device (saves the file)
  dev.off()

  message("Saved: ", filename)
}
```

The plots generated from the code chunk above will show the Gap statistics by the number of clusters for each dataset generated. As the range for the possible number of clusters is iterated through, a gap statistic is calculated for each potential number of clusters. This gap statistic compares the within cluster dispersion in the data and compares it to that of randomized data. The number of clusters associated with the largest value of the gap statistic is the optimal number of clusters supported by the dataset.

Plotting Estimated Number of Clusters by Side Length

To create the estimated number of clusters by side_length plot, I will first divide df_list into lists of datasets by their values of n. Then, I will run the clusGap function on each individual list, saving the optimal number of clusters for each dataset. Lastly, I will generate five plots, one per n value.

```r
D2 <- df_list[grepl("^df_2_", names(df_list))]
D3 <- df_list[grepl("^df_3_", names(df_list))]
D4 <- df_list[grepl("^df_4_", names(df_list))]
D5 <- df_list[grepl("^df_5_", names(df_list))]
D6 <- df_list[grepl("^df_6_", names(df_list))]
```

```r
estimate_clusters <- function(df_list, n_value) {

  results <- list()

  for (name in names(df_list)) {
    df <- df_list[[name]]
```

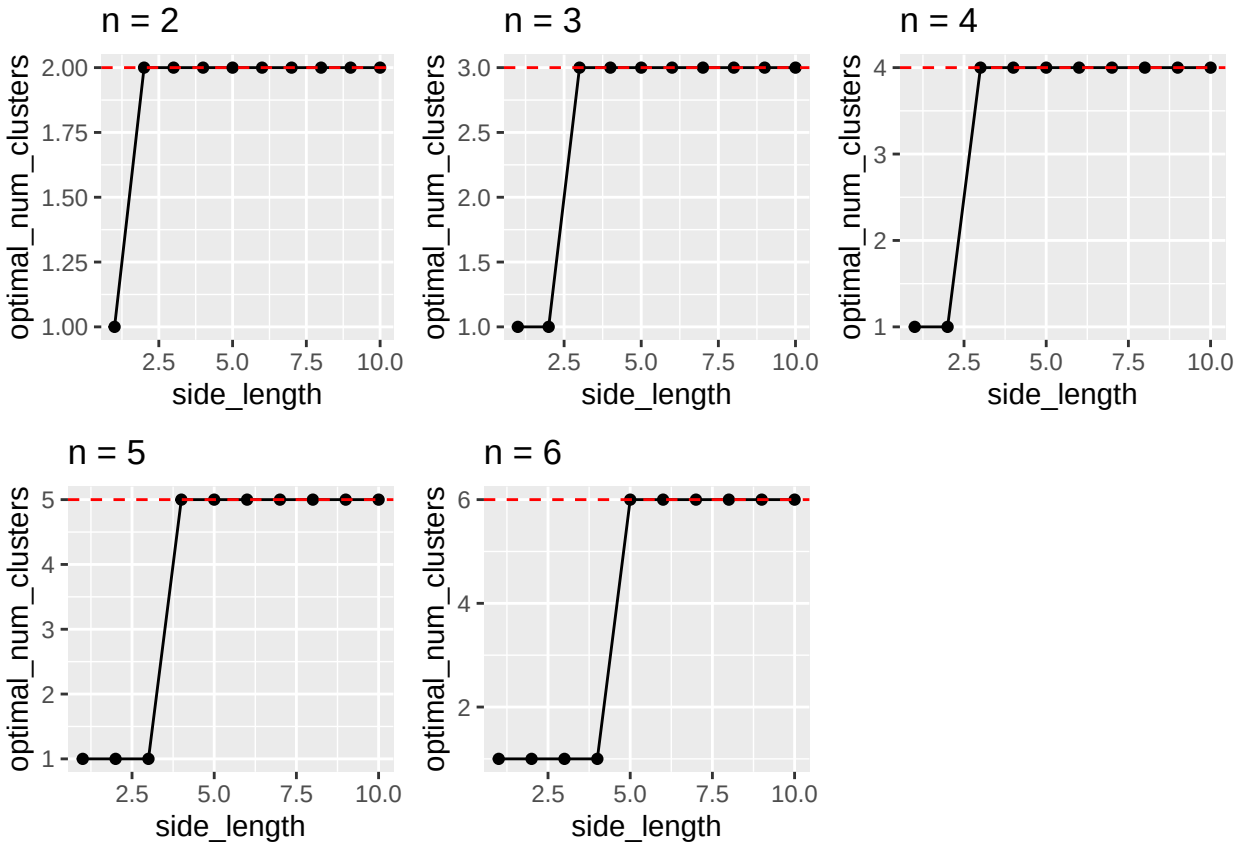
```
    # Drop label column if it exists
    if ("label" %in% names(df)) {
      df <- df %>% select(-label)
    }

    # Extract side_length from name (e.g., df_2_5 → 5)
    side_length <- as.integer(str_extract(name, "\\d+$"))

    # Run clusGap (you can increase B later)
    set.seed(123)

    gap <- clusGap(df, FUN = kmeans, K.max = n_value + 5, nstart = 20, iter.max = 50, B= 10)

    # Get optimal number of clusters using firstSEmax
    opt_nc <- maxSE(gap$Tab[,"gap"], gap$Tab[,"SE.sim"], method = "firstSEmax")

    # Store results
    results[[name]] <- tibble(
      dataset = name,
      n = n_value,
      side_length = side_length,
      optimal_num_clusters = opt_nc
    )
  }


  # Combine and return
  bind_rows(results)
}
```

```
gap_D2 <- estimate_clusters(D2, n_value = 2)
gap_D3 <- estimate_clusters(D3, n_value = 3)
gap_D4 <- estimate_clusters(D4, n_value = 4)
gap_D5 <- estimate_clusters(D5, n_value = 5)
gap_D6 <- estimate_clusters(D6, n_value = 6)
```

Now, I will first plot each gap statistic, and then I want to combine all of these ggplots into one figure for simplicity.

```
# Create individual plots
p2 <- ggplot(gap_D2, aes(x = side_length, y = optimal_num_clusters)) +
  geom_point() + geom_line() + ggtitle("n = 2") +
  geom_hline(yintercept = 2, linetype = "dashed", color = "red")

p3 <- ggplot(gap_D3, aes(x = side_length, y = optimal_num_clusters)) +
  geom_point() + geom_line() + ggtitle("n = 3") +
  geom_hline(yintercept = 3, linetype = "dashed", color = "red")

p4 <- ggplot(gap_D4, aes(x = side_length, y = optimal_num_clusters)) +
  geom_point() + geom_line() + ggtitle("n = 4") +
  geom_hline(yintercept = 4, linetype = "dashed", color = "red")

p5 <- ggplot(gap_D5, aes(x = side_length, y = optimal_num_clusters)) +
```

```
  geom_point() + geom_line() + ggtitle("n = 5")+
  geom_hline(yintercept = 5, linetype = "dashed", color = "red")

p6 <- ggplot(gap_D6, aes(x = side_length, y = optimal_num_clusters)) +
  geom_point() + geom_line() + ggtitle("n = 6")+
  geom_hline(yintercept = 6, linetype = "dashed", color = "red")

grid.arrange(p2, p3, p4, p5, p6, ncol = 3)
```



From these plots, we find that for n = 2, the optimal number of clusters is predicted is reduced after side_length = 1. For n = 3 and 4, the optimal number of clusters is reduced for side_length < 3. At n = 5, side_length < 4 is when the optimal number of clusters is reduced. Under n = 6, side_length < 5 is when the estimated number of clusters drastically reduces.

To save it separately as a figure:

```
if (!dir.exists("figures")) {
  dir.create("figures")
}

png("figures/cluster_gap_grid.png", width = 1000, height = 800)

grid.arrange(p2, p3, p4, p5, p6, ncol = 2)

invisible(dev.off())
```

Task 2: Spectral Clustering on Concentric Shells

This section will continue the work on task 2. The file data_generation_2.py has the beginning of this task, with the function for creating data, a test of the data generation function, an interactive plot of this sample data, and the creation as well as saving of the data sets with max radii from 10 to 0.

Within this section, I will first implement the function to transform the data such that the clusters are differentiable with kmeans. Then, I will implement the clusGap function to see at which point clusters are more challenging to distinguish. Lastly, I will generate a plot of the estimated number of clusters by the maximum radius allowed.

Data Loading:

```
file_list <- list.files("derived_data_2", pattern = "*.csv", full.names = TRUE)

# Read all CSVs into a named list of data frames
df_list2 <- file_list %>%
  set_names(~ basename(.) %>% tools::file_path_sans_ext()) %>%
  map(read_csv)
```

Spectral Clustering Transformation Function

```
# nested functions:

# Adjacency Matrix Creation -

adjmatrix <- function(data, d_threshold) {
  data <- as.matrix(data)

  dist_matrix <- as.matrix(dist(data, method = "euclidean"))
  adjacency_matrix <- ifelse(dist_matrix < d_threshold, 1, 0)

  diag(adjacency_matrix) <- 0 #doesn't make sense for a node to have a euclidean distance with itself

  return(adjacency_matrix)
}

# Normalized Laplacian Function -

Laplacian <- function(adj_matrix) {

  adj_matrix <- as.matrix(adj_matrix)
  degree_vector <- rowSums(adj_matrix)
  D <- diag(degree_vector)
  L <- D - adj_matrix

  # avoid division by zero
  inv_sqrt_deg <- 1 / sqrt(degree_vector)
  inv_sqrt_deg[!is.finite(inv_sqrt_deg)] <- 0  # set Inf or NaN to 0
```

```
  D_inv_sqrt <- diag(inv_sqrt_deg)

  # symmetric normalized Laplacian
  L_sym <- D_inv_sqrt %*% L %*% D_inv_sqrt

  return(L_sym)
}

# Eigendecomposition and Extracting the K Eigenvectors -

Eigenvectors <- function(matrix_input, k) {

  # Ensure input is numeric matrix
  matrix_input <- as.matrix(matrix_input)

  # Check if matrix is square
  if (nrow(matrix_input) != ncol(matrix_input)) {
    stop("Matrix must be square.")
  }

  eig <- eigen(matrix_input)

  idx <- order(eig$values)
  eigenvectors <- eig$vectors[, idx[1:k], drop = FALSE]


  # Return as numeric matrix
  return(eigenvectors)
}
```

Putting it all together:

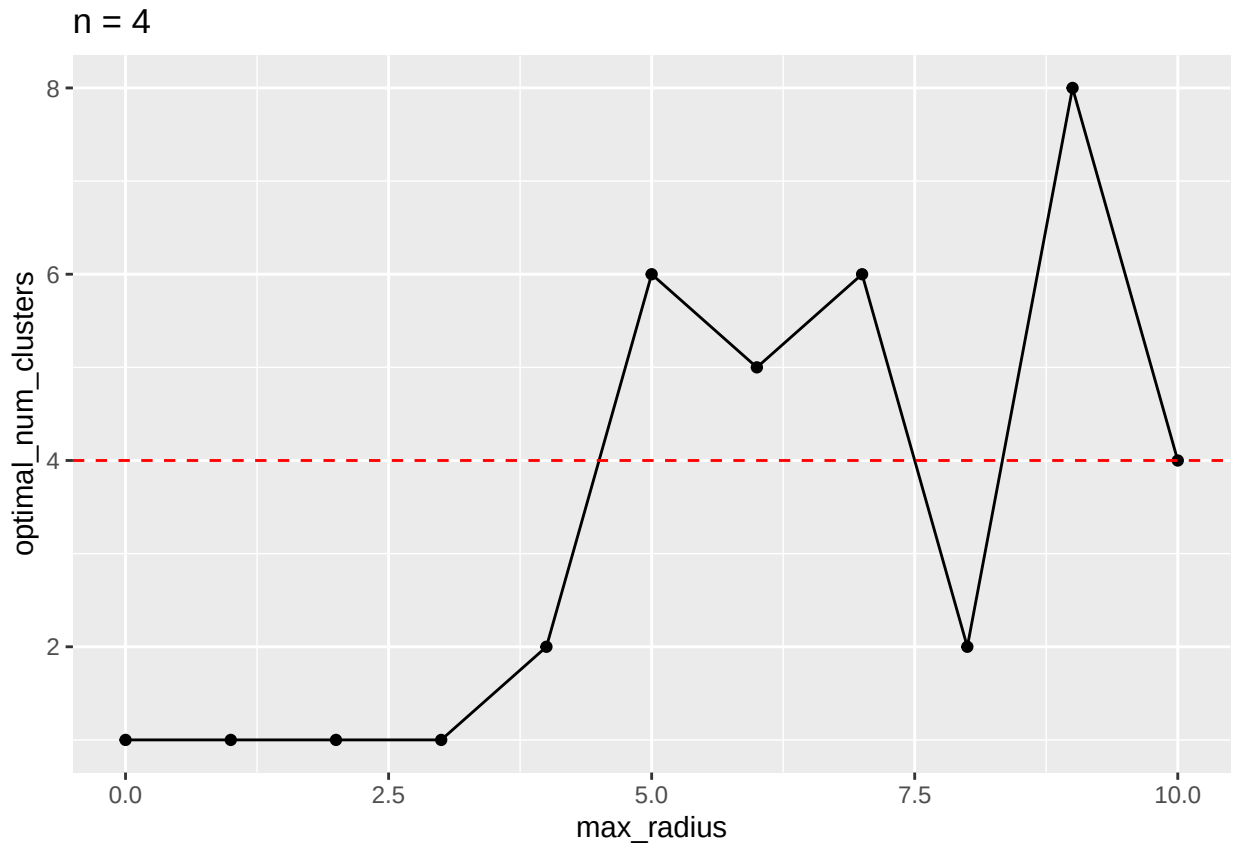
```
# Full Spectral Clustering Transformation Function -

Spectral <- function(x,k){
  # x is the dataset, k is the number of clusters (in our case, 4)
  A <- adjmatrix(x, 1)
  L <- Laplacian(A)
  V_set <- as.matrix(Eigenvectors(L, k))

  kmeans(V_set, k)
}
```

```
set.seed(123)
results <- list()

for (name in names(df_list2)) {
  df <- df_list2[[name]]
  gap <- clusGap(df, FUN = Spectral, K.max = 10, B= 10)
  opt_nc <- maxSE(gap$Tab[,"gap"], gap$Tab[,"SE.sim"], method = "firstSEmax")
  results[[name]] <- tibble(
    dataset = name,
    max_radius = as.integer(sub("df__", "", name)),
```

```
    optimal_num_clusters = opt_nc

  )
}

results2 <- bind_rows(results)
```

Plotting the Results:

```
ggplot(results2, aes(x = max_radius, y = optimal_num_clusters)) +
  geom_point() + geom_line() + ggtitle("n = 4")+
  geom_hline(yintercept = 4, linetype = "dashed", color = "red")
```



While the algorithm does not perform all that well, it seems as though the performance is particularly poor upon reaching a max_radius of 3 and below. I believe the algorithm would perform better if the d_threshold was decreased potentially, as that would force the identification of more tight-knit clusters and worse if the d_threshold were increased (would result in harder identification of clusters and more room for error).